

Guía de Consulta Práctica

Trabajo Práctico 03

Parte B - Planificación

1.-Diagrama de PERT

Propósito

COMPRENDER y APLICAR el diagrama de red (PERT) como herramienta para representar la secuencia lógica de tareas en un proyecto.

IDENTIFICAR el camino crítico y CALCULAR la duración total del mismo.

1.1 Conceptos Clave

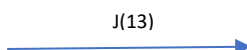
Concepto	Descripción
PERT (Program Evaluation and Review Technique)	Técnica de planificación que representa tareas como nodos conectados por relaciones de dependencia. Permite calcular fechas tempranas, tardías y holguras.
Camino crítico	Secuencia de tareas que determina la duración mínima del proyecto. No tiene holgura.
Fechas tempranas / tardías	Momentos más pronto o más tarde en que puede comenzar o terminar una tarea sin afectar el proyecto.
Holgura	Tiempo que una tarea puede retrasarse sin afectar la fecha final del proyecto.

1.2 Elementos Fundamentales

El diagrama de red PERT (Program Evaluation and Review Technique) representa gráficamente la secuencia lógica de tareas de un proyecto, permitiendo calcular tiempos, identificar dependencias y anticipar riesgos. A continuación, se detallan sus componentes clave:

1.2.1. Actividades (Tareas)

- **Definición:** Son las unidades de trabajo que deben realizarse en el proyecto.
- **Representación:** Se dibujan como flechas o arcos entre nodos (según convención).
- **Datos asociados:** Nombre o código de la tarea, duración estimada (en días), responsable, entregable.



1.2.2 Tarea Ficticia en el Diagrama de PERT

¿Qué es una tarea ficticia?

Una tarea ficticia (también llamada *actividad dummy*) es una conexión lógica entre dos eventos que no representa trabajo real ni consume tiempo, pero que es necesaria para mantener la coherencia del diagrama.

- Duración: siempre es cero
- Función: representar una dependencia lógica entre tareas
- Representación: flecha punteada o con etiqueta especial (por ejemplo, "F")
- No tiene entregables ni responsables asignados

¿Cuándo se usa?

Se utiliza cuando:

1. Dos tareas tienen la misma predecesora, pero una depende de otra y la otra no
2. Se necesita evitar ambigüedad en la secuencia de tareas
3. Se requiere mantener la unicidad de nodos (cada evento debe tener una única entrada y salida por tarea real)

🔍 Ejemplo Práctico

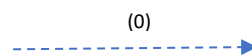
Supongamos estas tareas:

- T1: Relevamiento inicial
- T2: Modelado de clases (depende de T1)
- T3: Diagramas de secuencia (depende de T2)
- T4: Validación de requerimientos (también depende de T1, pero no de T2)

Para que el diagrama refleje correctamente que T3 depende de T2 y T4 solo de T1, se introduce una tarea ficticia entre los nodos de T1 y T4.

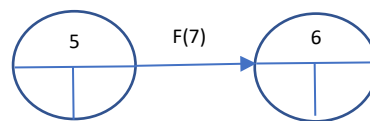
🧩 Cómo representarla:

- Se dibuja como una flecha punteada
- Se indica duración = 0
- Se coloca entre nodos para resolver ambigüedades sin alterar la lógica funcional



○ 1.2.3. Eventos (Nodos)

- **Definición:** Representan el inicio o finalización de una o más actividades.
- **Representación:** Se dibujan como círculos o rectángulos numerados. La numeración de los nodos no representa el orden de las tareas ni su secuencia temporal. Su única función es **identificar de manera única cada evento** dentro del diagrama, facilitando la construcción, lectura y análisis de la red lógica del proyecto.
- **Función:** Marcan puntos de control en el proyecto (inicio, bifurcación, convergencia).



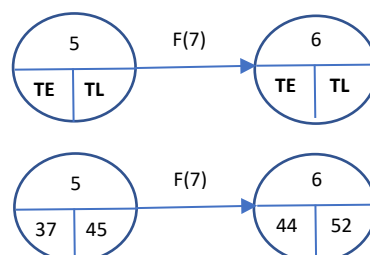
🔗 1.2.4. Dependencias

- **Definición:** Relación lógica entre tareas, que indica cuál debe completarse antes de iniciar otra.
- **Tipos:**
 - Fin–Inicio (la más común): una tarea comienza cuando otra termina.
 - Inicio–Inicio: dos tareas comienzan juntas.
 - Fin–Fin: dos tareas deben terminar al mismo tiempo.

⚙️ 1.2.5. Tiempos de ejecución

- **Duración estimada:** Tiempo que se espera que dure cada tarea.
- **Fecha temprana de inicio (TE):** El momento más pronto en que puede comenzar una tarea.
- **Fecha tardía de inicio (TL):** El momento más tarde en que puede comenzar sin retrasar el proyecto.
- **Holgura:** Diferencia entre TL y TE. Si es cero, la tarea es crítica.

Ejemplo:



1.2.6. Holgura (Slack)

- **Definición:** Tiempo que una tarea puede retrasarse sin afectar el proyecto.
- **Uso:** Permite reasignar recursos, ajustar cronograma o priorizar tareas.

Ejemplo: Si “Pruebas unitarias” tiene 2 días de holgura, puede comenzar más tarde si se necesita reforzar otra tarea.

1.2.7. Camino Crítico

- **Definición:** Secuencia de tareas que determina la duración mínima del proyecto.
- **Características:**
 - No tiene holgura.
 - Cualquier retraso en estas tareas afecta la fecha final.
 - Debe ser identificado y monitoreado.



2. Construcción y Análisis del Diagrama de PERT

2.1 - Parte 1 – Armado de la Red Técnica

🎯 Propósito

REPRESENTAR gráficamente la secuencia lógica de tareas de un proyecto, IDENTIFICANDO eventos de inicio y fin, dependencias funcionales y posibles paralelismos.

📖 Conceptos Clave

- **Tarea (actividad):** unidad de trabajo con duración definida.
- **Evento (nodo):** punto de inicio o finalización de una o más tareas.
- **Dependencia:** relación entre tareas que determina el orden de ejecución.
- **Tarea ficticia:** flecha sin duración que representa una dependencia lógica.

🔄 Procedimiento

1. **Identificar tareas y precedencias** desde la tabla del proyecto.
2. **Asignar nodos:** cada tarea tiene un nodo de inicio y uno de fin.
3. **Dibujar flechas:** cada flecha representa una tarea, conectando su nodo de inicio y fin.
4. **Agregar tareas ficticias** si es necesario para mantener la lógica de dependencias (por ejemplo, cuando dos tareas comparten predecesora, pero tienen relaciones distintas).

💡 *Cada nodo representa un evento, no una tarea. Las tareas se dibujan como flechas entre nodos.*

2.2 Parte 2 – Cálculo de Tiempos Tempranos (TE)

🎯 Propósito

DETERMINAR el momento más temprano en que puede comenzar y finalizar cada tarea, sin violar las dependencias.

📖 Conceptos Clave

- **TE inicio de nodo:** el mayor TE de los nodos que lo preceden.
- **TE fin de tarea:** TE del nodo de inicio + duración de la tarea.

🔄 Procedimiento

1. **Asignar TE = 0** al nodo inicial del proyecto.
2. **Avanzar de izquierda a derecha**, sumando la duración de cada tarea al TE del nodo de inicio.
3. **Cuando varias tareas llegan a un mismo nodo**, se toma el **mayor TE** entre ellas.
4. **Las tareas ficticias** no suman duración, pero pueden modificar el TE del nodo si son necesarias para mantener la lógica.

💡 *El TE del último nodo indica el tiempo total mínimo del proyecto.*

2.3 - Parte 3 – Cálculo de Tiempos Tardíos (TL)

🎯 Propósito

DETERMINAR el momento más tardío en que puede comenzar y finalizar cada tarea sin afectar la duración total del proyecto.

📖 Conceptos Clave

- **TL del nodo final** = TE del nodo final (tiempo total del proyecto).
- **TL de inicio de tarea** = TL del nodo de fin – duración de la tarea.
- **TL de nodo** = menor TL de las tareas que parten de él.

🔄 Procedimiento

1. **Asignar TL al nodo final igual al TE final.**
2. **Retroceder de derecha a izquierda**, restando la duración de cada tarea al TL del nodo de fin.
3. **Cuando varias tareas parten de un mismo nodo**, se toma el **menor TL** entre ellas.
4. **Las tareas ficticias** también se consideran en este cálculo para mantener la lógica de dependencias.

💡 *El TL permite identificar cuánto puede retrasarse una tarea sin afectar el proyecto.*

📅 2.4- Parte 4 – Cálculo de Holgura

🎯 Propósito

DETERMINAR el margen de tiempo disponible para cada tarea sin afectar la fecha final del proyecto.

📖 Conceptos Clave

- **Holgura** = $TL - TE$ del nodo de inicio de la tarea.
- Si la holgura es **0**, la tarea es **crítica**.

🔄 Procedimiento

1. Para cada tarea, calcular la diferencia entre TL y TE del nodo de inicio.
2. Registrar la holgura en la tabla de tareas.
3. Identificar las tareas con holgura positiva (pueden retrasarse) y las críticas (no pueden retrasarse).

💡 *La holgura permite tomar decisiones de asignación de recursos y ajustes de cronograma.*

🎯 2.5- Parte 5 – Identificación del Camino Crítico

🎯 Propósito

DETECTAR la secuencia de tareas que determina la duración mínima del proyecto y que no puede retrasarse.

📖 Conceptos Clave

- **Camino Crítico**: conjunto de tareas con holgura cero.
- **Impacto**: cualquier retraso en estas tareas afecta la fecha final del proyecto.

🔄 Procedimiento

1. Identificar todas las tareas con holgura = 0.
2. Verificar que estén conectadas secuencialmente en el diagrama.
3. Marcar visualmente el camino crítico (por ejemplo, con color rojo o doble línea).
4. Justificar por qué ese camino es el más largo y qué decisiones se deben tomar para protegerlo.

💡 *El camino crítico debe ser monitoreado durante todo el proyecto.*

■ 3.- Aplicación Práctica de la Guía de Apoyo: Construcción de un Diagrama PERT

A continuación, se presenta un caso práctico que permite aplicar los lineamientos metodológicos desarrollados en la *Guía de Apoyo – Construcción y análisis del Diagrama de PERT*. El objetivo es facilitar la apropiación conceptual del modelo PERT a través de una situación concreta, que incluye la identificación de tareas, el análisis de precedencias y la representación gráfica de la red lógica del proyecto.

Este ejercicio busca no sólo ilustrar el procedimiento técnico, sino también promover la comprensión de los fundamentos que sustentan el enfoque PERT: la planificación orientada a eventos, la gestión de dependencias y la detección del camino crítico.

Para el desarrollo del ejemplo, se utilizará una tabla de actividades en la que cada tarea ha sido identificada mediante una letra. Esta codificación permite simplificar la representación en el diagrama y focalizar el análisis en la estructura lógica del proyecto, sin perder de vista su aplicabilidad a contextos reales. A partir de esta base, se avanzará en la construcción del diagrama PERT siguiendo los pasos definidos en la guía.

Tarea	Descripción de la actividad en desarrollo de software	Tiempo (días)	Precedente(s)/ Dependencias
A	Definición del alcance y objetivos del proyecto. Reunión con stakeholders, análisis preliminar.	10	—
B	Levantamiento y análisis de requerimientos. Identificación de funcionalidades, restricciones y criterios de aceptación.	15	A
C	Diseño arquitectónico del sistema. Selección de patrones, definición de componentes y diagramas UML.	12	B
D	Diseño de interfaz de usuario (UI/UX). Prototipos, wireframes y validación con usuarios.	9	B
E	Configuración de infraestructura y entorno de desarrollo. Servidores, repositorios, CI/CD.	26	B
F	Desarrollo del backend. Implementación de lógica de negocio, APIs y persistencia.	7	C
G	Desarrollo del frontend. Integración con backend, componentes visuales y navegación.	4	C
H	Pruebas de usabilidad y ajustes de interfaz. Validación con usuarios reales, mejoras iterativas.	18	D
I	Integración y pruebas funcionales. Validación de flujos completos, pruebas de sistema.	15	F, H
J	Pruebas unitarias y revisión de código. Validación técnica, refactorización y documentación.	13	F, G
K	Pruebas de aceptación y validación final. Revisión con cliente, checklist de entregables.	10	I
L	Entrega y cierre del proyecto. Deploy final, capacitación, documentación y retrospectiva.	2	K

3.1 Parte 1 – Armado de la Red Técnica

3.1.1.-Identificar Tareas y Precedencias

El primer paso para construir un diagrama PERT consiste en identificar las tareas que conforman el proyecto y analizar sus relaciones de precedencia. Esta etapa es fundamental porque permite estructurar el proyecto como una secuencia lógica de actividades interdependientes, donde cada tarea representa una unidad funcional que contribuye al avance del sistema. En el caso presentado, se dispone de una tabla de actividades codificadas con letras, cada una con su duración estimada y una descripción contextualizada en el proceso de desarrollo de software. Para realizar este paso, se debe revisar cada tarea, registrar su duración, identificar las tareas de las que depende y comprender su función dentro del flujo de trabajo.

La tabla presenta 12 tareas codificadas con letras (A a L), cada una con su duración estimada y una descripción contextualizada en el ciclo de desarrollo de software. Para avanzar en el modelado, se recomienda construir una lista de dependencias entre las tareas identificadas. Este análisis permite establecer con claridad qué actividades deben completarse antes de que otras puedan comenzar, lo que resulta esencial para organizar la secuencia lógica del proyecto. Al definir las relaciones de precedencia, se determina el momento habilitado para el inicio de cada tarea, evitando solapamientos indebidos y asegurando que los recursos se asignen de manera eficiente. En el caso del desarrollo de software, por ejemplo, no se puede iniciar el diseño arquitectónico sin haber concluido el análisis de requerimientos, ni realizar pruebas funcionales sin haber integrado previamente los componentes desarrollados.

Esta lista de dependencias constituye la base para construir la red lógica del diagrama PERT, ya que permite visualizar el flujo de trabajo, identificar tareas que pueden ejecutarse en paralelo y detectar aquellas que condicionan el avance del proyecto. Además, facilita la posterior identificación del camino crítico, al evidenciar qué tareas tienen impacto directo en la duración total del proyecto.

Para realizar este paso, se recomienda:

1. Leer cada fila de la tabla y registrar:

- El nombre de la tarea (letra).
- Su duración (en días).
- Las tareas precedentes (si las hay).
- La descripción funcional (qué representa en el proceso).

2. Construir una lista de dependencias:

- A: no tiene precedentes → inicia el proyecto.
- B: depende de A → análisis de requerimientos.
- C, D, E: dependen de B → diseño técnico, interfaz e infraestructura.
- F, G: dependen de C → desarrollo backend y frontend.
- H: depende de D → pruebas de interfaz.
- I: depende de F y H → integración y pruebas funcionales.
- J: depende de F y G → revisión técnica.
- K: depende de I → validación final.
- L: depende de K → entrega y cierre.

Como recomendación adicional, una vez construida la lista de dependencias, es útil revisar si existen **relaciones múltiples que convergen en una misma tarea o divergencias que no pueden representarse directamente** en el diagrama sin generar ambigüedad. En estos casos, puede ser necesario incorporar **tareas ficticias**: actividades sin duración que permiten representar dependencias lógicas sin alterar el cálculo de tiempos. Estas tareas son especialmente útiles cuando se requiere distinguir entre caminos paralelos que comparten nodos o cuando una tarea depende de varias anteriores que no están conectadas entre sí. Detectarlas en esta etapa evita errores en la construcción de la red y garantiza la coherencia del modelo.

3.1.2.- Asignar nodos

El segundo paso en la construcción del diagrama PERT consiste en asignar nodos a cada tarea del proyecto. En este modelo, cada actividad se representa como una flecha que conecta dos nodos: uno de inicio y otro de fin. Los nodos no representan tareas, sino eventos que marcan el comienzo o la finalización de una o más actividades. Esta asignación permite construir la red lógica del proyecto, donde el flujo de trabajo se organiza en función de los eventos que habilitan o concluyen tareas.

Para cada actividad identificada en la tabla, se debe establecer un nodo de inicio (evento que habilita la tarea) y un nodo de fin (evento que marca su término). Por ejemplo, si la tarea B depende de A, el nodo de fin de A será el mismo que el nodo de inicio de B. Esta lógica garantiza la coherencia del modelo y permite representar con precisión las relaciones de precedencia.

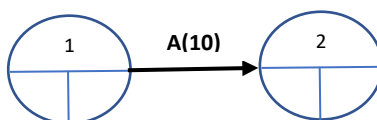
En casos donde una tarea depende de múltiples actividades que no comparten el mismo nodo de fin, puede ser necesario incorporar tareas ficticias para mantener la estructura lógica sin alterar los tiempos. Estas tareas no tienen duración, pero permiten representar dependencias complejas de forma clara y funcional.

La correcta asignación de nodos es esencial para avanzar en los cálculos de tiempos tempranos y tardíos, y para identificar el camino crítico del proyecto. Además, facilita la representación gráfica del diagrama, asegurando que cada actividad esté conectada de manera precisa dentro de la red.

Una vez construida la lista de dependencias en el paso anterior, se representa gráficamente esa estructura lógica mediante el diagrama PERT. Cada relación de precedencia definida se traduce en una conexión entre nodos, y cada tarea se ubica como una flecha entre eventos, respetando el orden y las condiciones de inicio. Esta visualización permite comprender de forma integral el flujo del proyecto, identificar bifurcaciones, convergencias y posibles tareas críticas, y constituye la base para los cálculos temporales que se desarrollarán en los siguientes pasos.

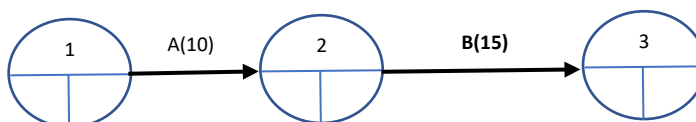
Para ello, se tomará cada entrada de la lista de tareas y se asignarán sus respectivos nodos de inicio y fin, respetando las relaciones de precedencia establecidas. Este recorrido permitirá visualizar progresivamente la red lógica del proyecto, asegurando que cada conexión refleje con precisión el flujo de trabajo definido en la etapa anterior.

A: no tiene precedentes → inicia el proyecto



B: depende de A → análisis de requerimientos

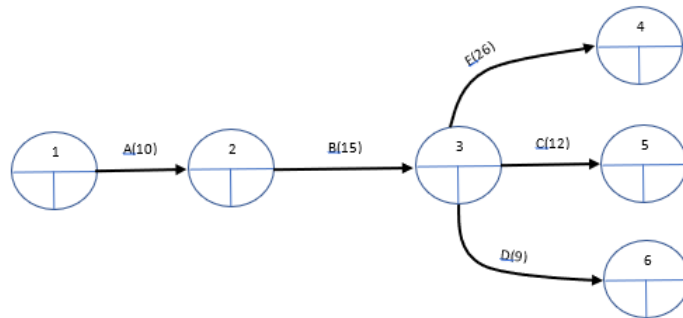
Esto implica que la ejecución de la tarea B no puede comenzar hasta que A haya finalizado. En el diagrama PERT que estamos construyendo, esto se traduce en que la tarea B debe iniciar en el mismo nodo donde termina la tarea A. Si asignamos el nodo 1 como el punto de inicio de A y el nodo 2 como su punto de finalización, entonces el nodo de inicio de B será también el nodo 2. Esta asignación garantiza la coherencia lógica del modelo y refleja correctamente la relación de precedencia entre ambas actividades.



C, D, E: dependen de B → diseño técnico, interfaz e infraestructura

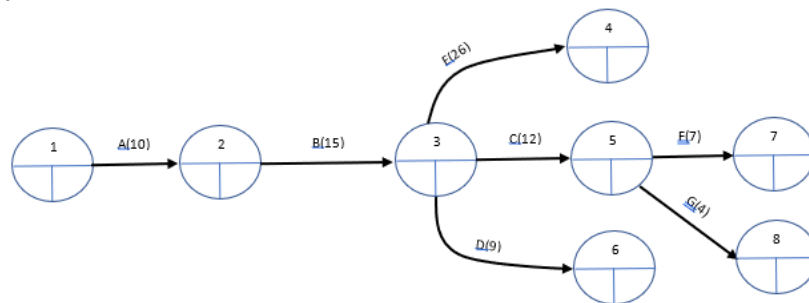
Las tareas C, D y E dependen directamente de la finalización de la tarea B, lo que significa que no pueden comenzar hasta que se haya completado el análisis de requerimientos. En el diagrama PERT, esta relación se traduce en que todas ellas comparten el mismo nodo de inicio, correspondiente al evento que marca el fin de B. Si asignamos el nodo 3 como punto de finalización de B, entonces C, D y E comenzarán en ese mismo nodo 3. Cada una de estas tareas tendrá su propio nodo de fin, ya que representan procesos distintos: el diseño técnico del sistema (C), el diseño de interfaz de usuario (D) y la configuración de infraestructura (E).

(E). Esta asignación permite representar correctamente la bifurcación del proyecto en tres líneas de trabajo paralelas que se habilitan simultáneamente tras el cierre de la etapa de análisis.



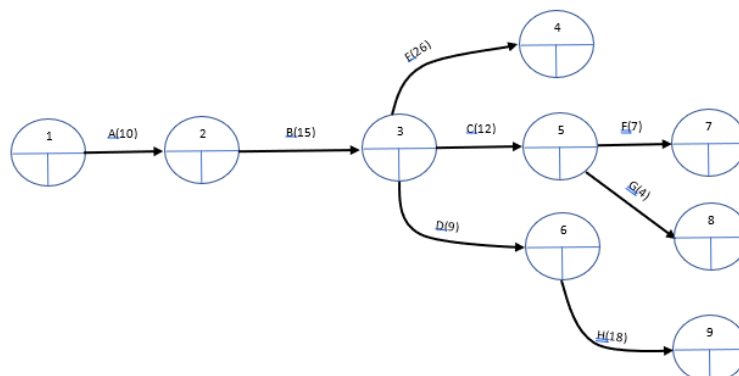
F, G: dependen de C → desarrollo backend y frontend

Las tareas F y G dependen directamente de la finalización de la tarea C, que corresponde al diseño arquitectónico del sistema. Esto significa que no pueden comenzar hasta que se haya completado dicha etapa técnica. En el diagrama PERT, esta relación se traduce en que ambas tareas comparten el mismo nodo de inicio, correspondiente al evento que marca el fin de C. Si asignamos el nodo 5 como punto de finalización de C, entonces tanto F como G comenzarán en ese nodo 5. Cada una tendrá su propio nodo de fin, ya que representan desarrollos distintos: F se enfoca en la implementación del backend (lógica de negocio, APIs, persistencia), mientras que G aborda el desarrollo del frontend (interfaz visual, navegación, integración). Esta bifurcación refleja dos líneas de trabajo paralelas que se habilitan simultáneamente tras el cierre del diseño técnico, y permite modelar con precisión la distribución de tareas en el equipo de desarrollo.



H: depende de D → pruebas de interfaz

La tarea H depende directamente de la finalización de la tarea D, que corresponde al diseño de la interfaz de usuario. Esto significa que las pruebas de interfaz no pueden comenzar hasta que el diseño haya sido completado y validado. En el diagrama PERT, esta relación se traduce en que H debe iniciar en el mismo nodo donde termina D. Si asignamos el nodo 6 como punto de finalización de D, entonces el nodo de inicio de H será también el nodo 6. Esta asignación refleja una secuencia lógica en el flujo de trabajo: primero se diseña la interfaz y luego se realizan las pruebas de usabilidad y ajustes necesarios. Al mantener esta conexión directa entre eventos, se garantiza que el modelo represente con precisión la dependencia funcional entre ambas tareas.



I: depende de F y H → integración y pruebas funcionales

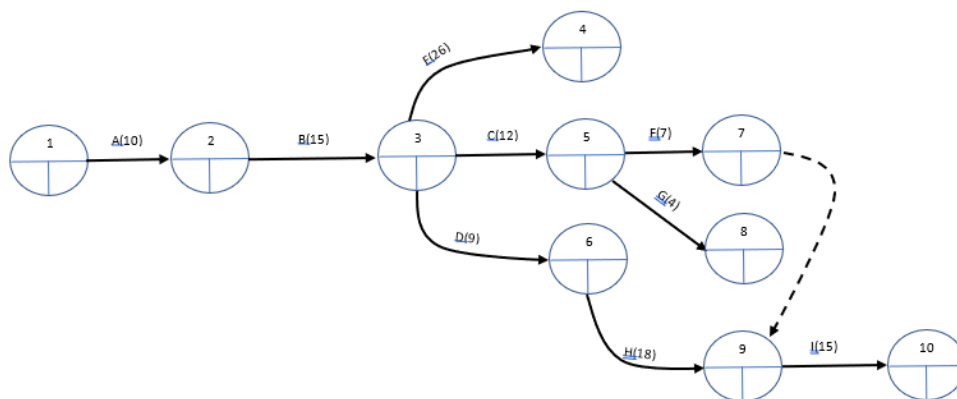
La tarea I depende de la finalización de dos actividades: F (desarrollo del backend) y H (pruebas de interfaz). En el diagrama PERT, esto implica que I debe comenzar una vez que ambas tareas hayan concluido. Sabemos que F finaliza en el nodo 7 y H en el nodo 9. Como estos nodos no coinciden, es necesario resolver esta convergencia para que I tenga un único nodo de inicio.

Una solución posible es crear un nuevo nodo (por ejemplo, el nodo 10) como punto de inicio de I, y conectar los nodos 7 y 9 a ese nuevo nodo mediante tareas ficticias (dummy). Esta opción se utiliza especialmente cuando **ambas tareas —F y H— son precedentes de otras actividades además de I**, ya que permite mantener la independencia lógica de cada rama del proyecto.

Sin embargo, en este caso particular, la tarea F sí es precedente de otra actividad (J), mientras que H no figura como precedente exclusivo de ninguna otra tarea; su única participación como antecedente es en combinación con F para habilitar la tarea I. Por lo tanto, se puede optar por una solución más eficiente: **trazar una tarea ficticia desde el nodo 7 (fin de F) hacia el nodo 9 (fin de H)**. Esto permite que I comience en el nodo 9, donde confluyen ambas dependencias. Es importante aclarar que la dirección de la tarea ficticia debe respetar la lógica de precedencia: al conectar el nodo 7 al nodo 9, se está indicando que F debe finalizar antes de que I pueda comenzar, junto con H.

Este tipo de decisión debe analizarse cuidadosamente al momento de construir el diagrama, ya que permite **evitar la incorporación innecesaria de nodos adicionales** que podrían complejizar la red sin aportar valor funcional. Siempre que una de las tareas convergentes no sea precedente exclusivo de otra actividad, conviene evaluar si una tarea ficticia dirigida entre nodos existentes resuelve la dependencia de forma más clara y eficiente. Esta práctica contribuye a mantener el modelo compacto, legible y fiel a la lógica del proyecto.

Este tipo de decisión en el modelado permite mantener la red clara y funcional, evitando la proliferación de nodos innecesarios y asegurando que las dependencias se representen con precisión. Además, refuerza la comprensión de cuándo conviene utilizar tareas ficticias simples y cuándo es necesario recurrir a nodos auxiliares para resolver convergencias complejas.



J: depende de F y G → revisión técnica

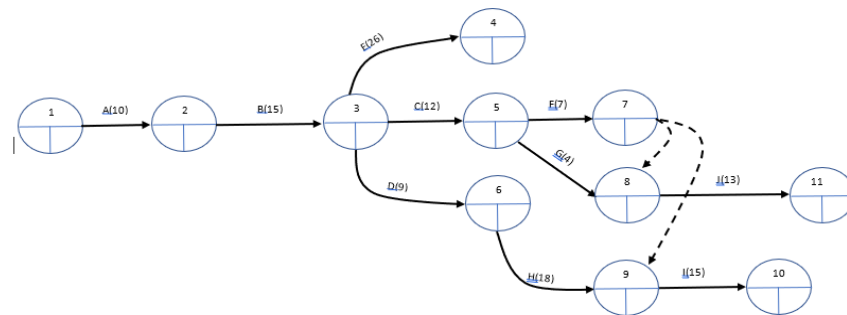
La tarea J depende de la finalización de dos actividades: F (desarrollo del backend) y G (desarrollo del frontend). En el diagrama PERT, esto implica que J debe comenzar una vez que ambas tareas hayan concluido. Sabemos que F finaliza en el nodo 7 y G en el nodo 8. Como estos nodos no coinciden, es necesario resolver esta convergencia para que J tenga un único nodo de inicio.

Una solución posible sería crear un nuevo nodo (por ejemplo, el nodo 10) como punto de inicio de J, y conectar los nodos 7 y 8 a ese nuevo nodo mediante tareas ficticias. Esta opción se utiliza cuando **ambas tareas convergentes son precedentes exclusivos de otras actividades**, ya que permite mantener la independencia lógica de cada rama.

Sin embargo, en este caso particular, la tarea F sí es precedente de otra actividad (I), mientras que G **no figura como antecedente exclusivo en ninguna otra relación**; su única participación como precedente es

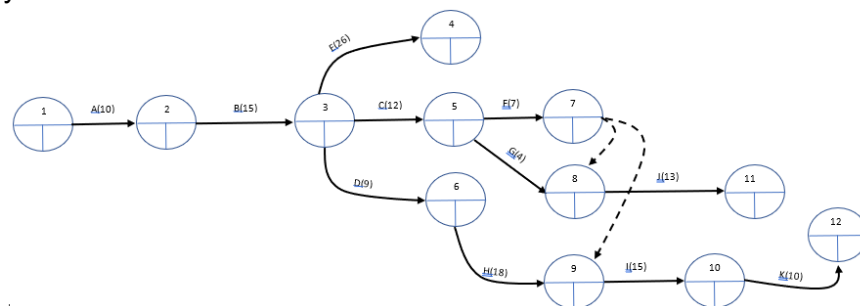
en combinación con F para habilitar la tarea J. Por lo tanto, se puede optar por una solución más eficiente: **trazar una tarea ficticia desde el nodo 8 (fin de G) hacia el nodo 7 (fin de F)**. Esto permite que J comience en el nodo 7, donde ya se encuentra la salida de F. La dirección de la tarea ficticia debe respetar la lógica de precedencia: al conectar el nodo 8 al nodo 7, se está indicando que G debe finalizar antes de que J pueda comenzar, junto con F.

Este tipo de decisión debe analizarse cuidadosamente al momento de construir el diagrama, ya que permite evitar la incorporación innecesaria de nodos adicionales que podrían complejizar la red sin aportar valor funcional. Siempre que una de las tareas convergentes no sea precedente exclusivo de otra actividad, conviene evaluar si una tarea ficticia dirigida entre nodos existentes resuelve la dependencia de forma más clara y eficiente. Esta práctica contribuye a mantener el modelo compacto, legible y fiel a la lógica del proyecto.



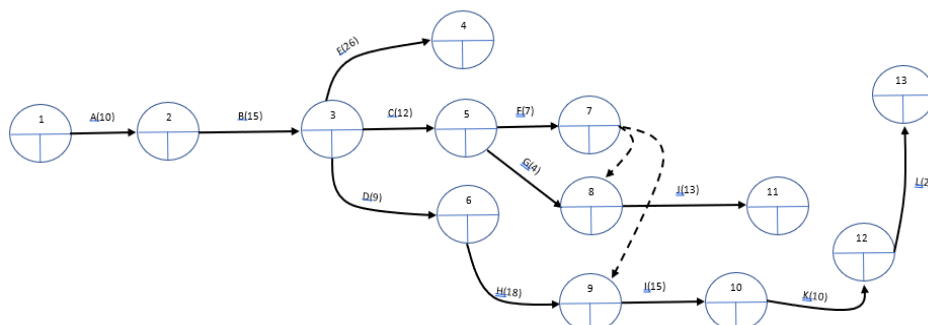
K: depende de I → validación final

La tarea K depende directamente de la finalización de la tarea I, que corresponde a la integración y pruebas funcionales. En el diagrama PERT, esta relación se representa de forma directa: el nodo de fin de I será el nodo de inicio de K. Si I finaliza en el nodo 10, entonces K comenzará en ese mismo nodo 10. Esta asignación refleja una secuencia lineal sin bifurcaciones ni convergencias, lo que simplifica la construcción de esta parte de la red lógica del proyecto.



L: depende de K → entrega y cierre

La tarea L depende directamente de la finalización de la tarea K, que corresponde a la validación final del sistema. En el diagrama PERT, esta relación se representa de forma directa: el nodo de fin de K será el nodo de inicio de L. Si K finaliza en el nodo 12, entonces L comenzará en ese mismo nodo 12. Esta asignación refleja una secuencia lineal que marca el tramo final del proyecto, donde la entrega y cierre se habilitan una vez completada la validación técnica.

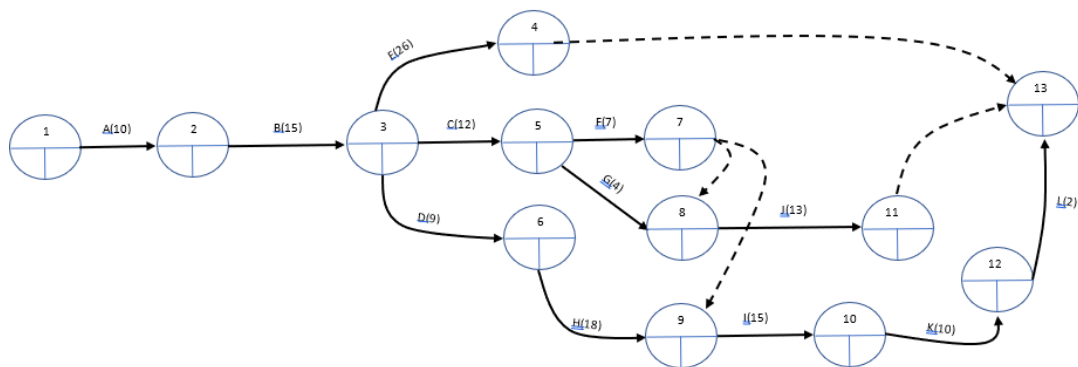


En la construcción del diagrama PERT, es importante asegurar que la red tenga **un único nodo de fin**, lo que permite definir con claridad el cierre del proyecto y facilita los cálculos de tiempos totales. Para lograrlo, se deben identificar los **nodos que no tienen sucesores**, es decir, aquellos que no habilitan ninguna tarea posterior.

Existen dos alternativas válidas para consolidar estos nodos finales:

1. **Conectar todos los nodos sin sucesores a un nuevo nodo de cierre** mediante tareas ficticias. Esta opción permite crear un nodo final explícito que agrupa todas las ramas del proyecto.
2. **Conectar directamente los nodos sin sucesores al último nodo existente** del diagrama, si ya se ha definido como cierre lógico del proyecto. Esta alternativa evita agregar nodos adicionales y puede ser más eficiente si el nodo final ya cumple esa función.

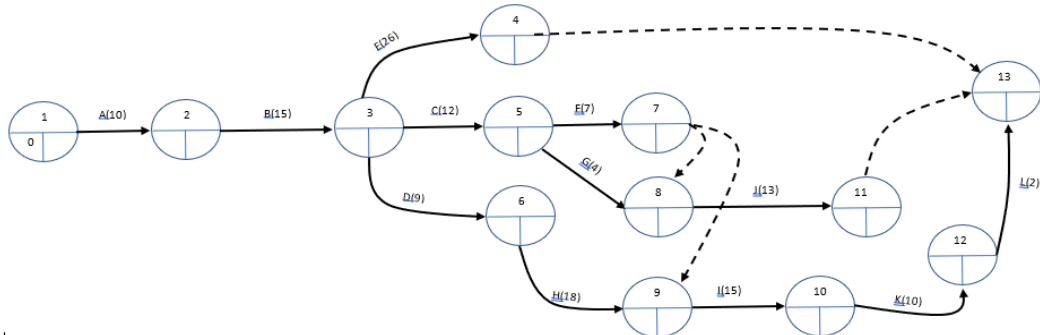
En el ejemplo que estamos desarrollando, los nodos 4 (fin de la tarea H) y 11 (fin de la tarea L) no tienen sucesores. Para unificar el cierre, se recomienda **trazar tareas ficticias desde ambos hacia un nodo final común**, que puede ser el nodo 13. Esta decisión permite que el diagrama tenga un único punto de término, manteniendo la red compacta y funcional.



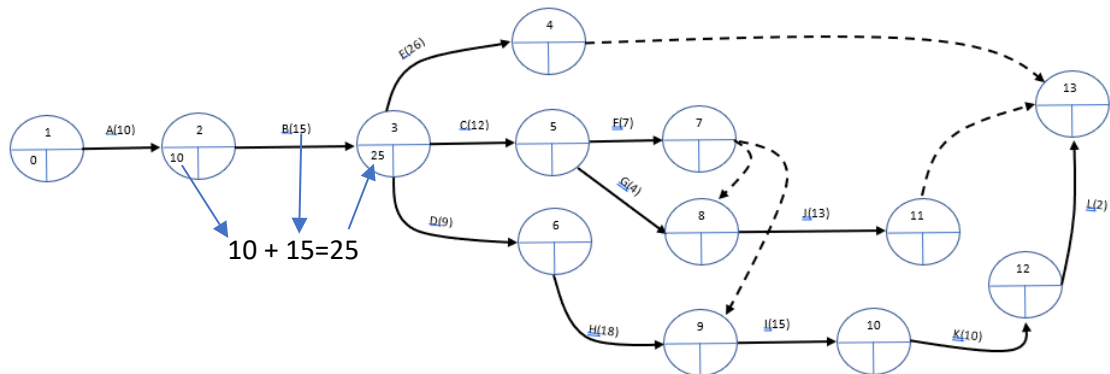
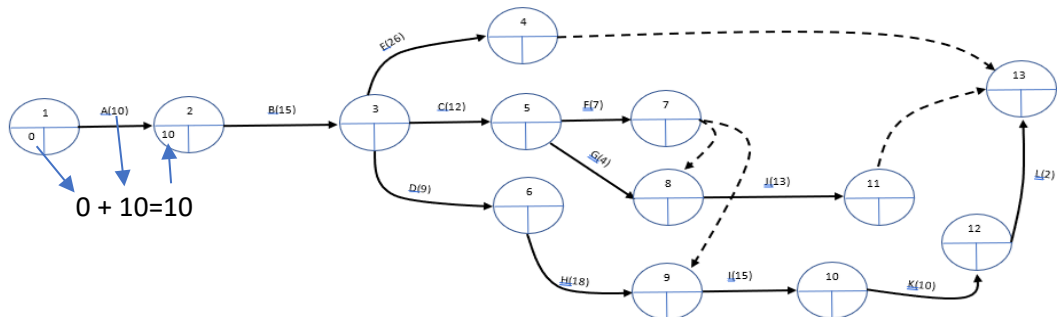
3.2 - Parte 2 – Cálculo de tiempos tempranos (TE)

Procedimiento

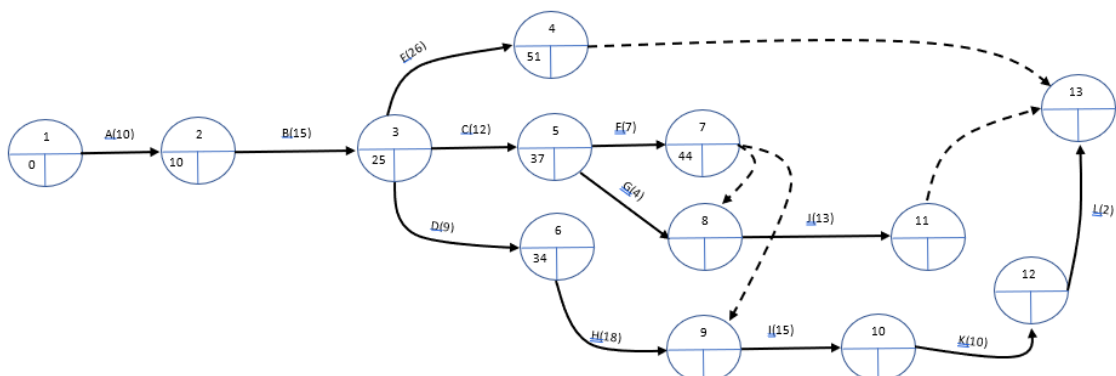
1. Asignar $TE = 0$ al nodo inicial del proyecto.



2. Avanzar de izquierda a derecha, sumando la duración de cada tarea al TE del nodo de inicio.



3. Cuando varias tareas llegan a un mismo nodo, se toma el mayor TE entre ellas, y 4. Las tareas ficticias no suman duración, pero pueden modificar el TE del nodo si son necesarias para mantener la lógica.



El nodo 8 presenta una situación típica de convergencia en el diagrama PERT: **se puede llegar a él por más de una tarea**, lo que obliga a aplicar el criterio de selección del mayor TE.

En este caso, el nodo 8 recibe:

- La tarea **G**, que parte del nodo 5 y tiene una duración de 4.
- Una **tarea ficticia** que parte del nodo 7, sin duración, utilizada para representar una dependencia lógica sin afectar los tiempos.

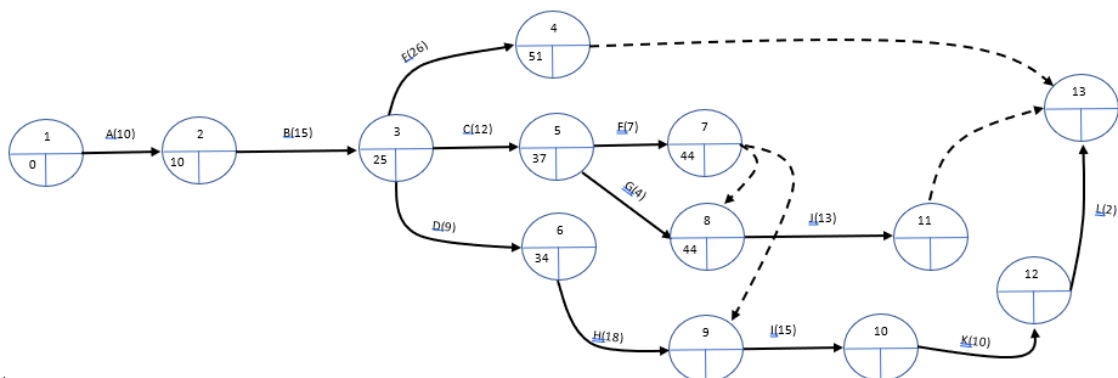
Esto significa que el nodo 8 puede habilitarse **una vez que se hayan completado cualquiera de esos caminos**, pero para respetar la lógica de sincronización del proyecto, **debe esperarse al más tardío de ellos**.

Para calcular el TE del nodo 8, se evalúan ambos caminos:

- Desde el nodo 5 por la tarea G:
 $TE_5 + \text{Duración G} = 37 + 4 = 41$
- Desde el nodo 7 por la tarea ficticia:
 $TE_7 + 0 = 44 + 0 = 44$

Como hay convergencia, se aplica la regla: **el TE del nodo 8 será el mayor de los tiempos que llegan por cada camino**:

$$TE_8 = \max(41, 44) = 44$$



Para el cálculo del **TE del nodo 9**, tenemos la misma situación que en el nodo 8: **dos caminos distintos que confluyen en un mismo nodo**, uno mediante una tarea real y otro mediante una tarea ficticia. Esto obliga a aplicar el criterio de selección del **mayor TE entre los caminos de llegada**, para garantizar que el nodo no se habilite hasta que todas las dependencias estén satisfechas.

El nodo 9 recibe:

- La tarea **H**, que parte del nodo 6 con una duración de **18**.
- Una **tarea ficticia** que parte del nodo 7, con duración **0**, utilizada para representar una dependencia lógica sin afectar los tiempos.

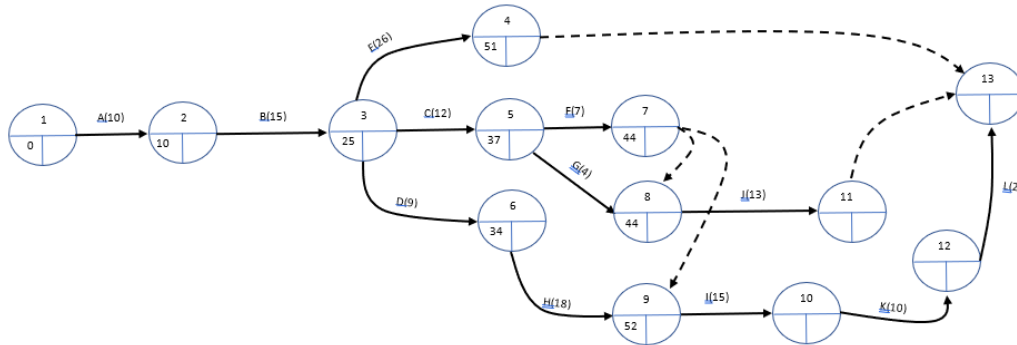
Para calcular el TE del nodo 9, se evalúan ambos caminos:

- Desde el nodo 6 por la tarea H:
 $TE_6 + \text{Duración H} = 34 + 18 = 52$
- Desde el nodo 7 por la tarea ficticia:
 $TE_7 + 0 = 44 + 0 = 44$

Como hay convergencia, se aplica la regla: **el TE del nodo 9 será el mayor de los tiempos que llegan por cada camino:**

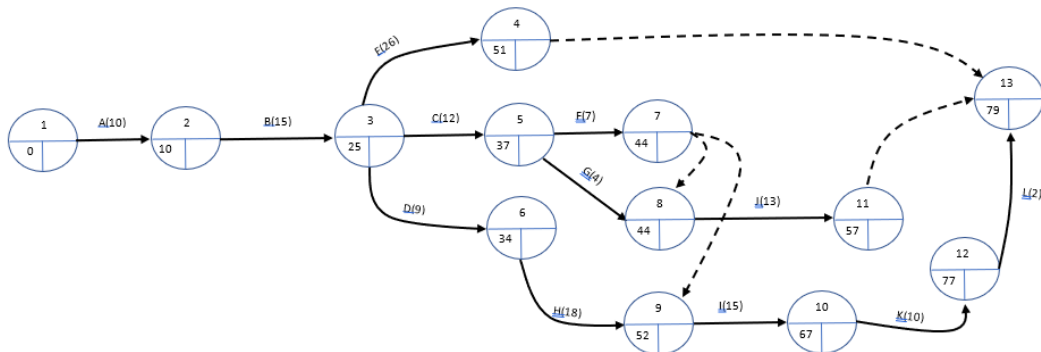
$$TE_9 = \max(52, 44) = 52$$

Este procedimiento asegura que el nodo 9 no se habilite hasta que **ambos caminos estén disponibles**, respetando la lógica de precedencias. Además, refuerza la comprensión de cómo las tareas ficticias pueden intervenir en la red sin alterar los tiempos, pero sí afectando la estructura lógica y la sincronización del proyecto.



Para completar el cálculo del **Tiempo más temprano (TE)** en el resto de los nodos del diagrama PERT, se deben seguir los pasos ya descritos. Este valor se determina en **todos los nodos**, ya sea que reciban una sola tarea o que actúen como punto de **convergencia de múltiples caminos**. En estos casos, se toma el **mayor TE** entre las tareas entrantes, asegurando que el nodo no se habilite hasta que **todas las actividades precedentes hayan finalizado**.

El **TE del último nodo** representa el **tiempo total del proyecto**, es decir, la duración mínima necesaria para completar todas las tareas planificadas sin interrupciones ni holguras. Este valor permite **identificar el camino crítico** y establecer el **horizonte temporal de ejecución**.

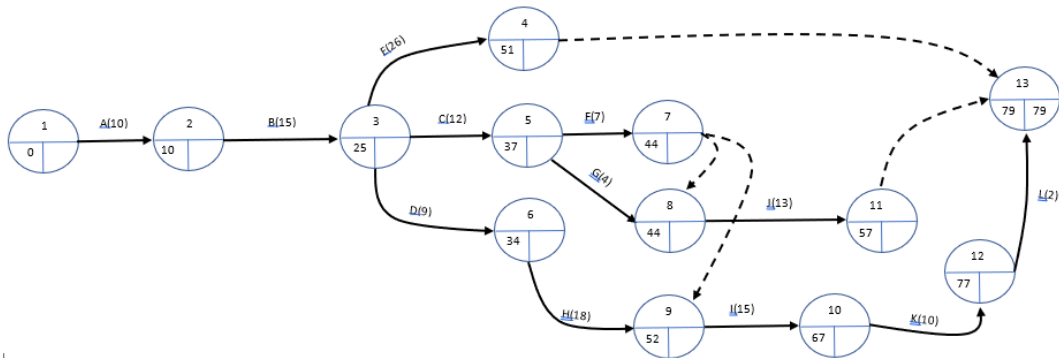


3.3 Parte 3 – Cálculo de tiempos tardíos (TL)

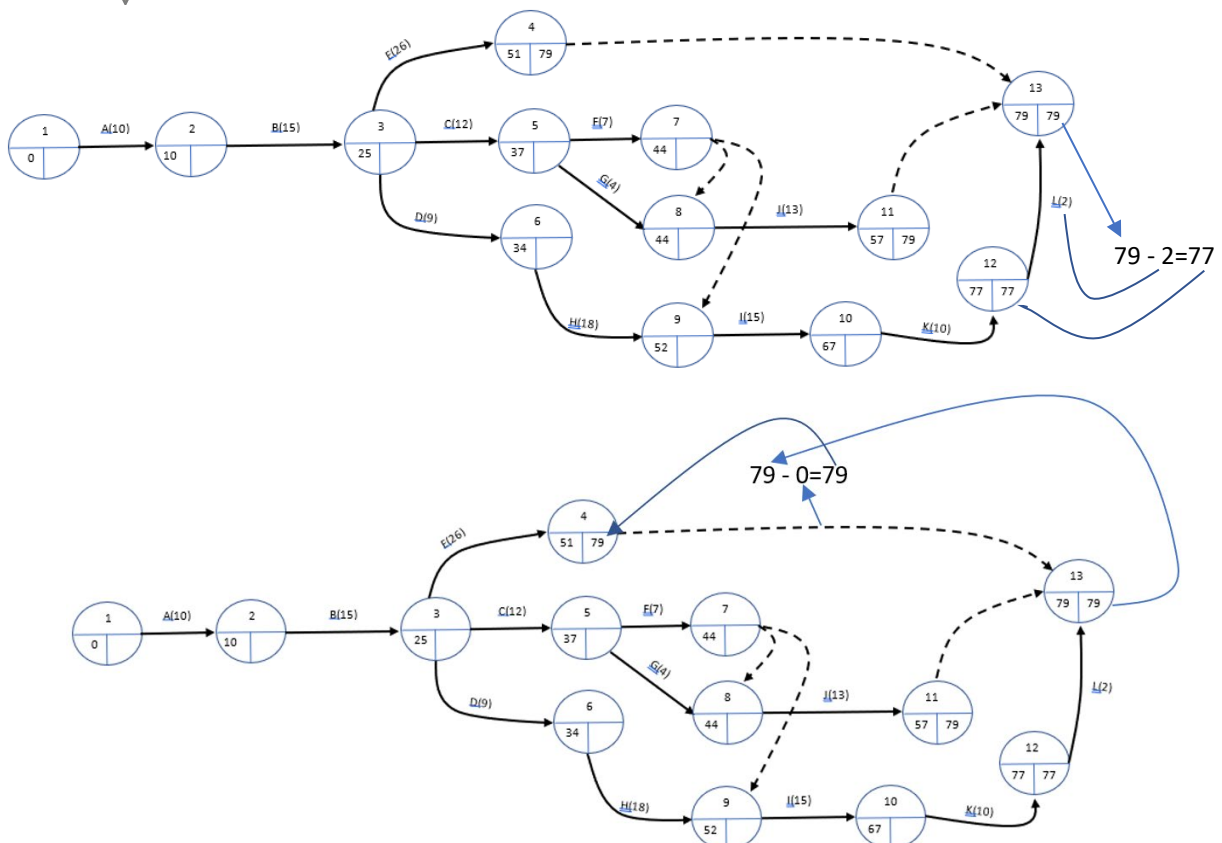
El **TL** de un nodo indica el momento más tardío en que puede ocurrir ese evento **sin retrasar el proyecto**, es decir, el instante límite en que puede iniciarse una tarea que parte desde ese nodo sin afectar el tiempo total. Se calcula **retrocediendo en la red**, desde el nodo final hacia el inicial, restando las duraciones de las tareas a medida que se recorren los caminos.

Procedimiento

1. Asignar TL al nodo final igual al TE final.



Retroceder desde el fin al inicio (derecha a izquierda), restando la duración de cada tarea al TL del nodo de **fin.**



2. Cuando varias tareas parten de un mismo nodo, se toma el **menor TL** entre ellas, y 4. Las **tareas ficticias** también se consideran en este cálculo para mantener la lógica de dependencias.

Si **varias tareas** salen del mismo nodo (es decir, hay divergencia), se deben calcular todos los posibles TL que se derivan por cada camino, y luego **tomar el menor de ellos**. Esto garantiza que el nodo se active **lo más tarde posible sin generar retrasos** en ninguna de las tareas que dependen de él.

El nodo 7 tiene **dos tareas salientes**, ambas ficticias:

- Una tarea ficticia que va al **nodo 9**, cuyo TL es **52**.
- Otra tarea ficticia que va al **nodo 8**, cuyo TL es **66**.

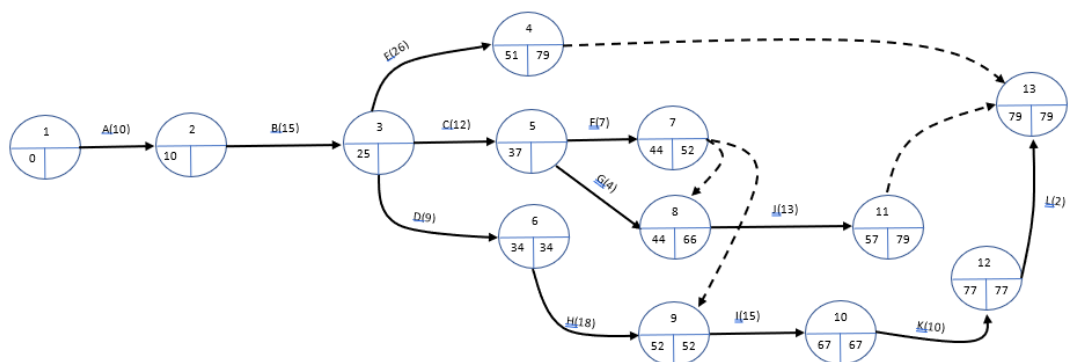
Ambas tareas tienen duración **0**, por lo tanto:

- Desde el nodo 9:
 $TL_9 - 0 = 52$
- Desde el nodo 8:
 $TL_8 - 0 = 66$

Como hay divergencia, se aplica la regla: **el TL del nodo 7 será el menor de los tiempos que permiten iniciar las tareas salientes sin retrasar el proyecto**:

$$TL_7 = \min(52, 66) = 52$$

Este procedimiento asegura que el nodo 7 no se active más tarde de lo permitido por las tareas que dependen de él, respetando la sincronización inversa del proyecto. Además, permite visualizar cómo las tareas ficticias, aunque no afectan los tiempos directamente, sí intervienen en la lógica de cálculo del TL.



Para el cálculo del **TL del nodo 5**, tenemos la misma situación que en el nodo 7: **dos caminos distintos que parten desde un mismo nodo**, uno mediante una tarea real y otro mediante una tarea que también condiciona el flujo posterior. Esto obliga a aplicar el criterio de selección del **menor TL entre los caminos de salida**, para garantizar que el nodo no se active más tarde de lo permitido por las tareas que dependen de él.

El nodo 5 tiene **dos tareas salientes**:

- La tarea **F**, que va al **nodo 7**, con una duración de **7** y TL del nodo 7 igual a **52**.
- La tarea **G**, que va al **nodo 8**, con una duración de **4** y TL del nodo 8 igual a **66**.

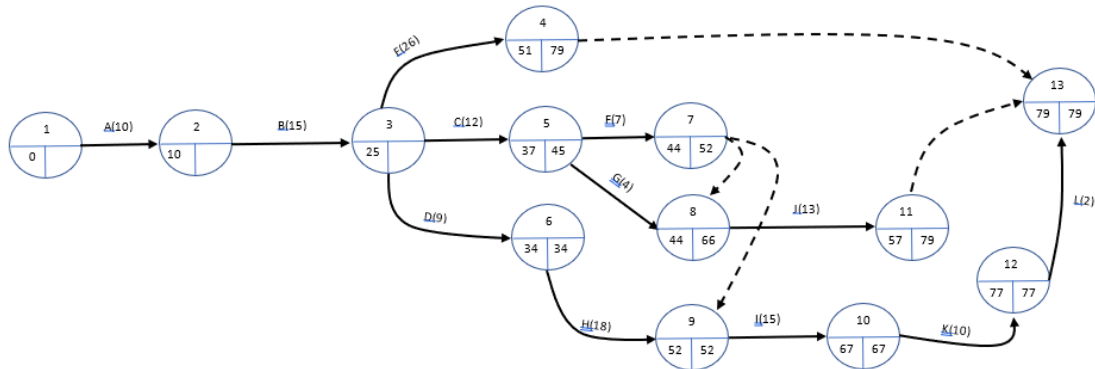
Esto significa que el nodo 5 debe habilitarse **lo suficientemente temprano** como para permitir que ambas tareas se inicien sin retrasar el proyecto. Para ello, se calcula el TL que permitiría iniciar cada tarea a tiempo, y luego se toma el **menor de esos valores**.

- Desde el nodo 7 por la tarea F:
 $TL_7 - \text{Duración F} = 52 - 7 = 45$
- Desde el nodo 8 por la tarea G:
 $TL_8 - \text{Duración G} = 66 - 4 = 62$

Como hay divergencia, se aplica la regla: **el TL del nodo 5 será el menor de los tiempos que permiten iniciar las tareas salientes sin retrasar el proyecto**:

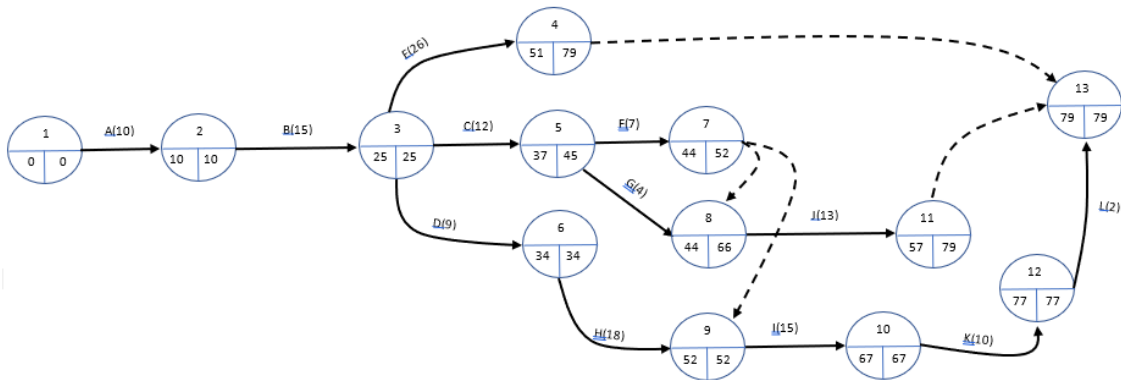
$$TL_5 = \min(45, 62) = 45$$

Este cálculo garantiza que el nodo 5 no se active más tarde de lo permitido por las tareas que dependen de él, respetando la lógica de sincronización inversa del proyecto. Además, refuerza la comprensión de cómo el TL se ajusta para proteger la integridad del cronograma en situaciones de divergencia.



Para completar el cálculo del **Tiempo más tardío (TL)** en el resto de los nodos del diagrama PERT, se deben seguir los pasos ya descritos, pero en sentido inverso al cálculo del TE. Este valor se determina en todos los nodos, ya sea que tengan una sola tarea saliente o que actúen como punto de divergencia hacia múltiples caminos. En estos casos, se toma el **menor TL** entre las tareas salientes, asegurando que el nodo no se active más tarde de lo permitido por las actividades que dependen de él.

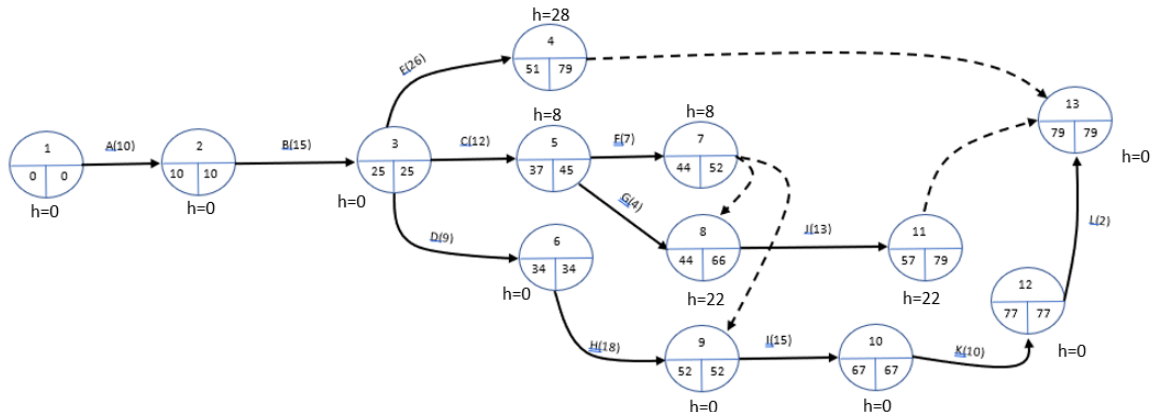
El recorrido se realiza desde el **último nodo hacia el primero**, y el cálculo debe finalizar en el **nodo 1**, al que se le asigna un **TL = 0**, ya que representa el inicio del proyecto y no puede retrasarse sin afectar la planificación global. Este valor permite identificar el camino crítico y verificar la consistencia temporal del cronograma.



3.4 - Parte 4 – Cálculo de holgura

Procedimiento

1. Para cada tarea, calcular la diferencia entre TL y TE del nodo de inicio.
2. Registrar la holgura en la tabla de tareas.
3. Identificar las tareas con holgura positiva (pueden retrasarse) y las críticas (no pueden retrasarse).



3.5 - Parte 5 – Identificación del camino crítico

Procedimiento

1. Identificar todas las tareas con holgura = 0.
2. Verificar que estén conectadas secuencialmente en el diagrama.
3. Marcar visualmente el camino crítico (por ejemplo, con color rojo o doble línea).
4. Justificar por qué ese camino es el más largo y qué decisiones se deben tomar para protegerlo.

El **camino crítico** es la **secuencia de tareas que determina la duración total del proyecto**. Está compuesto por aquellas actividades que **no tienen holgura**, es decir, que **no pueden retrasarse sin afectar la fecha final**. Cualquier demora en estas tareas implica una demora directa en el proyecto.

Una vez calculados los **Tiempos más temprano (TE)** y **Tiempos más tardío (TL)** para todos los nodos, se procede a:

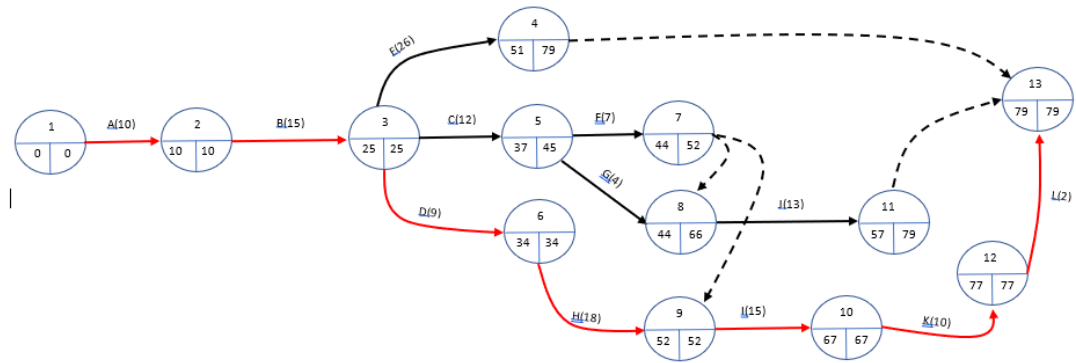
1. **Calcular la holgura total** de cada tarea:

La holgura total de una **tarea** se calcula con la siguiente expresión:

$$\text{Holgura} = \text{TL}_{\text{fin}} - \text{TE}_{\text{inicio}} - \text{Duración}$$

Donde:

- **TL_{fin}**: Tiempo más tardío del nodo donde **termina** la tarea.
 - **TE_{inicio}**: Tiempo más temprano del nodo donde **comienza** la tarea.
 - **Duración**: Tiempo que tarda en completarse la tarea (el valor asignado en el diagrama).
2. **Identificar las tareas con holgura igual a cero**. Estas tareas forman parte del camino crítico.
 3. **Recorrer la red siguiendo las tareas sin holgura**, desde el nodo inicial hasta el nodo final.



El camino crítico:

- Define la **duración mínima** del proyecto.
- Permite **priorizar recursos y seguimiento** sobre las tareas clave.
- Ayuda a **identificar riesgos**: cualquier retraso en estas actividades impacta directamente en el plazo global.

Tareas del camino crítico: **A – B – D – H – I – K – L**

4.- Conclusión

A partir del desarrollo completo del diagrama PERT, se logró calcular los **Tiempos más temprano (TE)** y **Tiempos más tardío (TL)** en cada nodo, identificar las **holguras** de las tareas y determinar el **camino crítico** del proyecto. Este recorrido permitió comprender cómo se articulan las dependencias, cómo se representan las tareas ficticias y cómo se interpreta la lógica temporal de ejecución.

El análisis realizado muestra que el camino crítico define la **duración mínima del proyecto**, y que cualquier retraso en las tareas que lo componen impacta directamente en el plazo final. La identificación de holguras también permite visualizar qué tareas tienen margen de maniobra y cuáles requieren seguimiento prioritario.

Este ejemplo puede tomarse como referencia para aplicar el procedimiento en otros casos, y como cierre del práctico, resume los pasos esenciales para construir, analizar e interpretar una red PERT completa.

5. - Marco Teórico – Diagrama de Gantt

El **Diagrama de Gantt** es una herramienta gráfica utilizada en la gestión de proyectos para representar visualmente el desarrollo temporal de las actividades. Fue creado por Henry L. Gantt a comienzos del siglo XX y se ha convertido en uno de los instrumentos más difundidos para planificar, coordinar y controlar tareas en función del tiempo.

A diferencia del **Diagrama PERT**, que se enfoca en la lógica de precedencias y utiliza unidades de tiempo abstractas (días, semanas, etc.), el diagrama de Gantt permite **asociar cada tarea a fechas reales**, facilitando la visualización del cronograma completo del proyecto.

5.1- Características Principales

- Representa las tareas como **barras horizontales** sobre un eje temporal.
- Muestra la **duración**, el **inicio** y el **fin** de cada actividad.
- Permite visualizar **superposiciones**, **secuencias** y **dependencias** entre tareas.
- Facilita el seguimiento del avance del proyecto y la **asignación de recursos** cuando se utiliza en herramientas digitales o software de gestión.

5.2 - Utilidad en Planificación

El Diagrama de Gantt es especialmente útil para:

- Traducir la estructura lógica del proyecto en un **calendario operativo**.
- Identificar **momentos clave**, **cuellos de botella** y **zonas de riesgo**.
- Coordinar equipos y recursos en función de fechas concretas.
- Comunicar el plan de trabajo de forma clara y accesible.

5.3 - Recomendación: Uso de Herramientas CASE

Para construir y gestionar diagramas de Gantt de forma eficiente, se recomienda el uso de **Herramientas CASE** (Computer-Aided Software Engineering), como Microsoft Project, GanttProject, Trello con extensiones, o plataformas educativas con módulos de planificación.

Estas herramientas permiten:

- **Asignar recursos** a cada tarea (personas, equipos, materiales).
- Establecer **dependencias automáticas** entre actividades.
- Visualizar el avance del proyecto con indicadores dinámicos.
- Generar **informes y alertas** para el seguimiento.
- Exportar el cronograma en formatos editables o compartibles.

El uso de Herramientas CASE no solo agiliza la construcción del diagrama, sino que también mejora la toma de decisiones, la comunicación entre equipos y la trazabilidad del proyecto.

🕒 6.- Guía para Construir el Diagrama de Gantt

El Diagrama de Gantt permite representar gráficamente el desarrollo temporal de las tareas de un proyecto, mostrando **cuándo comienza y termina cada actividad** en función de fechas reales o planificadas.

A diferencia del Diagrama PERT, que se basa en unidades de tiempo abstractas y relaciones de precedencia, el Gantt ofrece una **visualización cronológica** clara y directa.

📋 6.1. Reunir la Información Base

Antes de comenzar, asegurate de tener:

- El listado completo de tareas del proyecto.
- La duración de cada tarea.
- Las dependencias entre tareas (qué actividades deben completarse antes de iniciar otras).
- Las fechas de inicio previstas del proyecto.

📏 6.2. Definir el Eje Temporal

- El eje horizontal representa el **calendario del proyecto**: días, semanas o meses, según el nivel de detalle.
- El eje vertical muestra las **tareas**, ordenadas por código o por secuencia lógica.

📊 6.3. Representar las Tareas

- Para cada tarea, dibujá una **barra horizontal** que comience en la fecha de inicio prevista y se extienda según su duración.
- Las tareas que dependen de otras deben ubicarse después de la finalización de las actividades precedentes.
- Si se usa una herramienta CASE o software de gestión de proyectos, se pueden **asignar recursos** (personas, equipos, materiales) a cada tarea, lo que permite visualizar cargas de trabajo y disponibilidad.

🧠 6.4. Ventajas del Diagrama de Gantt

- Permite ver **fechas concretas** de inicio y fin de cada tarea.
- Facilita el seguimiento del avance del proyecto en tiempo real.
- Es compatible con herramientas digitales que permiten **asignar recursos**, establecer alertas y generar informes automáticos.
- Ofrece una visión clara para equipos de trabajo, responsables de planificación y seguimiento.

Este diagrama complementa el análisis PERT al traducir la lógica de precedencias en una **planificación calendarizada**, útil para la gestión operativa del proyecto.

7.- Herramientas CASE para diagramas de Gantt

7.1. GanttProject

- **Tipo:** Software libre y gratuito.
- **Plataforma:** Windows, macOS, Linux.
- **Características:**
 - Permite crear diagramas de Gantt con dependencias entre tareas.
 - Asignación de recursos y exportación en PDF o PNG.
 - Ideal para uso educativo y proyectos simples.
- Link: <https://www.ganttproject.biz/>

7.2. ProjectLibre

- **Tipo:** Gratuito y de código abierto.
- **Plataforma:** Windows, macOS, Linux.
- **Características:**
 - Alternativa libre a Microsoft Project.
 - Diagramas de Gantt, ruta crítica, asignación de recursos.
 - Compatible con archivos de MS Project.
- Link: <https://www.projectlibre.com/>

7.3. ClickUp

- **Tipo:** Freemium (versión gratuita con funciones limitadas).
- **Plataforma:** Web / App móvil / Escritorio.
- **Características:**
 - Gestión de tareas con vista Gantt interactiva.
 - Asignación de responsables, fechas, prioridades.
 - Ideal para trabajo colaborativo.
- Link: <https://clickup.com/>

7.4. Trello + Extensiones

- **Tipo:** Freemium.
- **Plataforma:** Web / App móvil.
- **Características:**
 - Trello por sí solo no tiene vista Gantt, pero se puede integrar con extensiones como **Planyway** o **TeamGantt**.
 - Gestión visual de tareas con cronograma.
 - Muy accesible para estudiantes.
- Link: <https://trello.com/>

7.5. TeamGantt

- **Tipo:** Freemium.
- **Plataforma:** Web.
- **Características:**
 - Interfaz intuitiva para crear diagramas de Gantt colaborativos.
 - Seguimiento de progreso, dependencias, carga de trabajo.
 - Integración con Trello, Slack, y otros.
- Link: <https://www.teamgantt.com/>

Bibliografía

- Sommerville, I. (2005). *Ingeniería de software* (7.^a ed.). Prentice Hall.
- Larman, C. (2005). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3rd ed.). Prentice Hall.
- Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
- Booch, G., Rumbaugh, J., & Jacobson, I. (1999). *The Unified Modeling Language User Guide*. Addison-Wesley.
- GanttProject. (s.f.). *GanttProject – Free Project Scheduling and Management Software*. Recuperado de <https://www.ganttproject.biz/>. 13/09/2025.
- Microsoft. (s.f.). *Microsoft Project – Project Management Software*. Recuperado de <https://www.microsoft.com/es-ar/microsoft-365/planner/microsoft-planner?ms.url=mscomproject>. 13/09/2025.
- ProjectLibre. (s.f.). *ProjectLibre – Open Source Project Management*. Recuperado de <https://www.projectlibre.com/>. 13/09/2025.
- ClickUp. (s.f.). *ClickUp – One App to Replace Them All*. Recuperado de <https://clickup.com/>. 13/09/2025.
- Trello. (s.f.). *Trello – Organize Anything, Together*. Recuperado de <https://trello.com/>. 13/09/2025.
- TeamGantt. (s.f.). *TeamGantt – Online Gantt Chart Software*. Recuperado de <https://www.teamgantt.com/>. 13/09/2025.